

UNIFEOB — Centro Universitário Octávio Bastos

Escola de Negócios e Tecnologia EAD — Análise e Desenvolvimento de Sistemas

PROJETO INTEGRADO

Redes de Computadores e Algoritmos

1º Trimestre Letivo de 2026

Diagnóstico e Melhoria da Rede Corporativa

Aluno: Matheus Francisco | RA: 24001882

Orientadora: Prof.^a Mariangela Martimbianco Santos

São João da Boa Vista, SP — Junho de 2026

1. Identificação do Projeto

Título	Diagnóstico e Melhoria da Rede Corporativa
Empresa	MF Tecnologia e Sistemas
Período	Maio – Junho de 2026
Aluno / RA	Matheus Francisco — RA 24001882
Orientadora	Prof. ^a Mariangela Martimbianco Santos
Algoritmo central	Dijkstra (caminho de menor latência) sobre grafo ponderado
Ferramentas	Python 3 · Cisco Packet Tracer · Relatório HTML com servidor local
Repositório	https://github.com/mathfrancisco/Projeto-Integrado-Redes-de-Computadores-e-Algoritmos
ODS Relacionado	ODS 9 — Indústria, Inovação e Infraestrutura (Agenda 2030/ONU)

2. Introdução e Objetivos

A **MF Tecnologia e Sistemas** opera com uma rede corporativa única e plana, na qual computadores administrativos, estações de desenvolvimento, servidores, dispositivos IoT e visitantes compartilham o mesmo domínio de broadcast. Essa ausência de segmentação cria riscos de segurança, gargalos de tráfego e pontos únicos de falha — um problema silencioso que só se manifesta sob carga ou incidente.

Este projeto modela a rede como um **grafo ponderado** (dispositivos = nós, conexões = arestas, latência = peso) e aplica o **algoritmo de Dijkstra** para encontrar caminhos de menor latência, identificar gargalos e validar uma arquitetura nova, segmentada por VLANs, com firewall e redundância de link. A conexão com o **ODS 9** é direta: infraestrutura segura, segmentada e resiliente sustenta inovação e continuidade do negócio.

Objetivos Específicos

- Inventariar dispositivos e medir latência/perda de pacotes (ping, traceroute).
- Representar a topologia atual e a proposta como grafos em JSON validados.
- Executar Dijkstra para comparar o custo DEV1 → CLOUD nas duas redes.
- Identificar nós com maior concentração de rotas (pontos críticos de falha).
- Projetar VLANs, regras de acesso e firewall, validando por matriz de testes.

3. Arquitetura de Rede

A rede atual concentra todo o tráfego em um único switch (SW1) e um único roteador de borda (R1) com link de internet único — qualquer falha nesses nós derruba toda a operação. A proposta introduz um switch core com VLANs, um firewall-roteador e dois links de operadora (ISP1/ISP2) para redundância.

Topologia atual (cenário simulado)

Dispositivo	Tipo	Setor	Conexão (latência)
DEV1	Computador	Desenvolvimento	→ SW1 (1,0 ms, Ethernet)
ADM1	Computador	Administração	→ SW1 (1,2 ms, Ethernet)
SRV1	Servidor	Servidores	→ SW1 (0,8 ms, Ethernet)
IOT1	IoT	IoT	→ AP1 (5,5 ms, Wi-Fi)
GUEST1	Computador	Visitantes	→ AP1 (7,0 ms, Wi-Fi)
AP1	Access Point	—	→ SW1 (1,5 ms, Ethernet)
SW1	Switch	—	→ R1 (2,0 ms, Ethernet)
R1	Roteador de borda	—	→ INTERNET (12,0 ms, WAN)
INTERNET / CLOUD	WAN / Nuvem	—	INTERNET → CLOUD (18,0 ms, WAN)

Topologia proposta (segmentada)

Dispositivo	Tipo	VLAN	Conexão (latência)
DEV1	Computador	20	→ SW_CORE (0,6 ms)
ADM1	Computador	10	→ SW_CORE (0,7 ms)
SRV1	Servidor	30	→ SW_CORE (0,4 ms)
IOT1	IoT	40	→ AP_IOT (2,0 ms) → SW_CORE (0,8 ms)
GUEST1	Computador	50	→ AP_GUEST (3,0 ms) → SW_CORE (0,9 ms)
SW_CORE	Switch core	Trunk	→ FW (0,8 ms, Trunk)
FW	Firewall-roteador	—	→ ISP1 (8,0 ms) · → ISP2 (12,0 ms)
ISP1 / ISP2	Links WAN	—	→ CLOUD (18,0 / 20,0 ms)

4. Modelagem em Grafo e Pensamento Computacional

A rede é carregada de um arquivo JSON para a classe Grafo (lista de adjacência ponderada). O algoritmo de **Dijkstra** usa uma fila de prioridade (heapq) para calcular as menores distâncias a partir de uma origem; `reconstruir_caminho()` rastreia os predecessores até montar a rota. Pesos negativos são rejeitados na validação, pois Dijkstra exige arestas não negativas.

Pilares do Pensamento Computacional

Pilar	Aplicação no Projeto
Decomposição	Solução dividida em módulos: <code>grafo.py</code> (estrutura), <code>dijkstra.py</code> (algoritmo), <code>analise.py</code> (comparação/gargalos), <code>main.py</code> (CLI).
Abstração	<code>Grafo.de_json()</code> esconde parsing e validação; <code>menor_caminho()</code> devolve um <code>ResultadoCaminho</code> sem expor a fila de prioridade.
Padrões	<code>concentracao_de_rotas()</code> identifica nós usados por mais caminhos mínimos — revela pontos únicos de falha (ex.: SW1, R1).
Algoritmo	Dijkstra com <code>heapq</code> — complexidade $O((V+E) \log V)$; descarta entradas obsoletas da fila comparando a distância acumulada.

Tabela de decisão do caminho (DEV1 → CLOUD)

Topologia	Caminho de menor latência	Custo total
Atual	DEV1 → SW1 → R1 → INTERNET → CLOUD	33,00 ms
Proposta	DEV1 → SW_CORE → FW → ISP1 → CLOUD	27,40 ms

5. Protocolos, Segmentação e Conectividade

A nova arquitetura separa a rede em VLANs por finalidade. Cada setor recebe uma faixa de IP própria e regras de acesso específicas no firewall, reduzindo a superfície de ataque e o domínio de broadcast. A medição de conectividade usa **ICMP** (ping/traceroute) e os caminhos lógicos são validados pelo Dijkstra.

VLAN	Setor	Rede	Gateway
10	Administração	192.168.10.0/24	192.168.10.1
20	Desenvolvimento	192.168.20.0/24	192.168.20.1

VLAN	Setor	Rede	Gateway
30	Servidores	192.168.30.0/24	192.168.30.1
40	IoT	192.168.40.0/24	192.168.40.1
50	Visitantes	192.168.50.0/24	192.168.50.1
99	Gerenciamento	192.168.99.0/24	192.168.99.1

Matriz de testes de acesso

ID	Origem	Destino	Esperado
T01	PC_ADMIN	SRV_APP	Permitido
T02	PC_DEV	SRV_APP	Permitido
T03	PC_DEV	PC_ADMIN	Bloqueado
T04	IOT1	SRV_MQTT	Permitido
T05	IOT1	PC_ADMIN	Bloqueado
T06	PC_GUEST	SRV_APP	Bloqueado
T07	PC_GUEST	CLOUD_SERVER	Permitido
T08	PC_DEV	CLOUD_SERVER	Permitido

6. Implementação e Código-Fonte

Repositório GitHub: github.com/mathfrancisco/Projeto-Integrado-Redes-de-Computadores-e-Algoritmos

Estrutura de Módulos

Arquivo	Responsabilidade
src/grafos.py	Classe Grafo (lista de adjacência), validação de nós/arestas e carga via de_json().
src/dijkstra.py	Algoritmo de Dijkstra com heapq, reconstruir_caminho() e menor_caminho().
src/analise.py	comparar_topologias() e concentracao_de_rotas() (gargalos).
main.py	CLI com subcomandos caminho, gargalos e comparar.
scripts/servidor_testes.py	Servidor local Python que expõe a API e serve o relatório HTML interativo.
tests/test_dijkstra.py	Testes automatizados do algoritmo e da comparação de topologias.

Exemplo de execução (CLI)

```
python main.py comparar --atual data/topologia_atual.json \
  --proposta data/topologia_proposta.json --origem DEV1 --destino CLOUD
```

TOPOLOGIA ATUAL

Caminho: DEV1 -> SW1 -> R1 -> INTERNET -> CLOUD

Custo: 33.00 ms

TOPOLOGIA PROPOSTA

Caminho: DEV1 -> SW_CORE -> FW -> ISP1 -> CLOUD

Custo: 27.40 ms

Diferença: 5.60 ms

Melhoria estimada: 16.97%

Formato da topologia (JSON)

```
{"nome": "...", "nos": [{"id": "DEV1", "tipo": "computador", "vlan": 20}],
  "arestas": [{"origem": "DEV1", "destino": "SW_CORE", "latencia_ms": 0.6, "meio": "Ethernet"}]}
```

7. Resultados, Segurança e Conclusão

Validação Funcional

Cenário	Resultado	Status
Dijkstra DEV1 → CLOUD (atual)	33,00 ms — rota via SW1/R1 (link único)	OK
Dijkstra DEV1 → CLOUD (proposta)	27,40 ms — rota via SW_CORE/FW/ISP1	OK
Melhoria de latência	5,60 ms mais rápido — 16,97% de redução	OK
Concentração de rotas (atual)	SW1 e R1 concentram a maioria dos caminhos = pontos únicos de falha	OK
Redundância de link (proposta)	Falha de ISP1 → roteamento por ISP2 mantém CLOUD acessível	OK
Segmentação por VLAN	Matriz de testes valida acessos permitidos e bloqueados	OK
Peso negativo na topologia	Rejeitado por ErroTopologia (Dijkstra exige pesos ≥ 0)	OK

Segurança e Escalabilidade

A rede atual, plana e sem firewall, permite tráfego lateral livre entre setores e depende de um único switch, roteador e link. A proposta isola setores em VLANs, aplica regras de acesso no firewall e adiciona um segundo link de operadora (ISP2) para redundância. A arquitetura escala adicionando novas VLANs/switches ao grafo sem alterar o algoritmo — basta atualizar o JSON da topologia.

Conclusão

O projeto cumpriu os requisitos do Projeto Integrado de **Redes de Computadores e Algoritmos**: modelagem da rede como grafo ponderado, aplicação do algoritmo de Dijkstra, identificação de gargalos e pontos únicos de falha, e validação de uma arquitetura segmentada e redundante. A proposta reduz a latência do caminho crítico em **16,97%**, elimina pontos únicos de falha e organiza a rede em VLANs — diretamente alinhada ao **ODS 9 — Indústria, Inovação e Infraestrutura**, com replicação imediata para outras unidades da MF Tecnologia e Sistemas.

8. Referências

CORMEN, T. H. et al. *Algoritmos: Teoria e Prática*. 4. ed. Rio de Janeiro: GEN LTC, 2022.

DIJKSTRA, E. W. A note on two problems in connexion with graphs. *Numerische Mathematik*, v. 1, 1959.

TANENBAUM, A. S.; WETHERALL, D. J. *Redes de Computadores*. 6. ed. São Paulo: Pearson, 2021.

IETF. RFC 792 — Internet Control Message Protocol (ICMP). 1981.

CISCO SYSTEMS. *Packet Tracer — Documentação e Laboratórios*. Disponível em: <https://www.netacad.com>

ONU. *Objetivos de Desenvolvimento Sustentável — ODS 9. Agenda 2030*. Disponível em: <https://brasil.un.org/pt-br/sdgs/9>